

set vs unordered_set & Vector Capacity

C++ Հարցազրույցի ամբողջական ուղեցույց

Այս ուղեցույցը ընդգրկում է STL-ի երկու ամենասիրած interview թեմա՝ `set` vs `unordered_set` (ներքին կառուցվածք, complexity, ordering, lower_bound), և vector-ի **size vs capacity vs reserve vs resize** (memory management, reallocation): Ամեն մեկը կդիմարարվի, բացատրված և interview հարցերով:

ՄԱՍ 1 — set vs unordered_set

set — Red-Black Tree (sorted)

```
set<int> s;
s.insert(30);
s.insert(10);
s.insert(20);

for (int x : s) cout << x << " "; // 10 20 30 (Միշտ sorted)
```

Ներքում` Balanced BST (Red-Black Tree): Տվյալները միշտ պահվում են **sorted** կարգով:

unordered_set — Hash Table (unordered)

```
unordered_set<int> us;
us.insert(30);
us.insert(10);
us.insert(20);

for (int x : us) cout << x << " "; // 20 30 10 կամ gankacac karg
```

Ներքում` Hash Table: Order-ի guarantee **չկա**:

Complexity

Գործօրօthyun	set	unordered_set
insert	$O(\log n)$	$O(1)$ average, $O(n)$ worst
find	$O(\log n)$	$O(1)$ average, $O(n)$ worst
erase	$O(\log n)$	$O(1)$ average, $O(n)$ worst
lower/upper_bound	$O(\log n)$	ՉկԱ
ordered traversal	$O(n)$ sorted	հնարavor չէ

Ordering

set — փշտ sorted.

```
insert 30,10,20 -> 10 20 30
```

unordered_set — ոչ sorted.

```
insert 30,10,20 -> 30 10 20 կամ այլ կարգ (hash-ից կախած)
```

Duplicate — Երկու-սի մեյ ՉԿԱ

Երկու container-ի էլ **unique** են (կրկնապիտակ չեն պահում).

```
set<int> s;  
s.insert(10);  
s.insert(10);  
s.insert(10);  
// s = {10} (միայն մեկ)
```

Նույնի **unordered_set**-ում.

lower_bound — միայն set-ում

```
set<int> s = {10, 20, 30};  
auto it = s.lower_bound(25); // araji element >= 25 -> 30
```

```
unordered_set<int> us;  
// us.lower_bound(25); // COMPILE ERROR -- sorted չէ
```

lower_bound / **upper_bound** -y աշխատում են միայն sorted container-ում (set), զանի որ hash-based container-ի կարգ չունի.

Կարևոր ֆունկցիաներ (set)

```
s.insert(10);
s.find(10);           // iterator kam end()
s.erase(10);
s.count(10);          // 0 kam 1 (unique)
s.lower_bound(10);    // araji >= 10
s.upper_bound(10);    // araji > 10
s.size();
s.empty();
```

Typical Problem — Duplicate Check

```
vector<int> nums = {1, 2, 3, 4, 2};
unordered_set<int> seen;
for (int x : nums) {
    if (seen.count(x)) { cout << "Duplicate"; break; }
    seen.insert(x);
}
// O(n) average
```

Երբ որն օգտագործել

set: sorted data, range queries, lower_bound/upper_bound, ordered traversal
unordered_set: fast lookup/insert/erase, only existence checking, order petq չէ (DEFAULT)

Interview պատասխան

set -> Balanced Tree (Red-Black), Sorted, $O(\log n)$
unordered_set -> Hash Table, Not Sorted, $O(1)$ average, $O(n)$ worst

ՄԱՍ 2 — Vector size vs capacity

Interview-նւ՛ շատ-շատ en harcnum:

size() — իրական էլեմենտների քանակ

```
vector<int> v;  
v.push_back(10);  
v.push_back(20);  
v.push_back(30);  
cout << v.size();           // 3
```

capacity() — hatkacvac memory

```
cout << v.capacity();       // 4 kam 8 (implementation-ic kaxvac)
```

size = քանի element ka; **capacity** = քանի element-i tex e ARDEN hatkacvac (sovorabar >= size):

Աճյ քայլ a՛ր քայլ

```
vector<int> v;                // size=0, capacity=0  
v.push_back(10);             // size=1, capacity=1  
v.push_back(20);             // size=2, capacity=2  
v.push_back(30);             // size=3, capacity=4 <- reallocation, x2
```

```
+-----+-----+-----+-----+  
| 10 | 20 | 30 |   |           size=3, capacity=4  
+-----+-----+-----+-----+  
                        ^ azat tex (capacity - size)
```

Inchu e capacity petq

Ete amen push_back-i hamar nor memory allocate aner, shat dandagh klinep (amen angam $O(n)$ copy): Dra hamar vector-y pahum e lracucich azat tex, u push_back-y amortized $O(1)$ e.

Reallocation

```
size=4, capacity=4: [1][2][3][4]
push_back(5):      tex chka ->
  1. nor memory allocate
  2. capacity mecacnum (x2 -> 8)
  3. bolor arjeqnery texapoxum (copy/move)
  4. hin memory azatum
Result:            [1][2][3][4][5]
```

Reallocation-y $O(n)$ e -> dra hamar karevor e reserve()-y.

reserve() — memory naxapatveluc

```
vector<int> v;
v.reserve(1000);           // size=0, capacity=1000
// 1000 push_back -> grethe voch mi reallocation
```

reserve(n) — capacity-n bardzracnum e minchev n, bayc **element chi steghcum** (size chi poxvum):

resize() — element-neri qanak poxelu

```
vector<int> v;
v.resize(100);             // size=100, capacity>=100
// 100 hat element STEGHCUM e (default 0-ov)
```

resize(n) — poxum e IRAKAN element-neri qanaky (size):

reserve() vs resize() (ωδenasirvac harc)

```
vector<int> a;
a.reserve(100); // size=0, capacity=100 -> element CHI steghcum
cout << a.size(); // 0

vector<int> b;
b.resize(100); // size=100, capacity>=100 -> 100 element steghcum (0-ov)
cout << b.size(); // 100
```

	reserve(n)	resize(n)
Uqđum e	capacity	size (u capacity ete petq)
Element steghcum?	ŋ	U3ŋ (default value)
size poxvum?	n	uŋo -> n
Npatak	reallocation-ic xusapel	element-ner unenal

shrink_to_fit()

```
v.shrink_to_fit(); // avelcuk capacity azatum (capacity -> size)
```

Interview հարցեր & պատասխաններ

set vs unordered_set?

set = Red-Black Tree, sorted, $O(\log n)$, lower_bound ka. unordered_set = Hash Table, unsorted, $O(1)$ average / $O(n)$ worst, lower_bound chka.

Erku-n el duplicate pahum en?

Voch, erku-n el unique en (key mek angam).

lower_bound vor set-um ka?

Miayn set-um (sorted). unordered_set-um compile error.

size vs capacity?

size = irakan element-neri qanak; capacity = hatkacvac memory ($\geq$ size).

reserve vs resize?

reserve = miayn capacity, element chi steghcum, size chi poxvum. resize = size poxum, element steghcum (default).

Inchu capacity petq e?

Amen push_back-i reallocation-ic xusapelu; azat tex -> amortized $O(1)$ push_back.

Reallocation inch e?

capacity lvelis nor mec memory, x2 cap, bolor element copy/move, hin azatum. $O(n)$.

push_back amortized complexity?

$O(1)$ amortized (x2 growth -> reallocation hazvadep).

Cheat Sheet

set (Red-Black Tree):

sorted | $O(\log n)$ | lower/upper_bound | unique | range query

unordered_set (Hash Table):

unsorted | $O(1)$ avg, $O(n)$ worst | unique | fast lookup | NO lower_bound (DEFAULT)

VECTOR CAPACITY:

size() = irakan element-ner

capacity() = hatkacvac memory ($\geq$ size)

reserve(n) = capacity $\rightarrow$ n (element CHI steghcum, size=nuyny)

resize(n) = size $\rightarrow$ n (element STEGHCUM, default value)

shrink_to_fit() = capacity $\rightarrow$ size (avelcuk azatum)

REALLOCATION: capacity full $\rightarrow$ new mem, x2, copy/move all, free old ($O(n)$)

$\rightarrow$ reserve() naxapes $\rightarrow$ xusapum

ONE-LINERS:

set = sorted tree $O(\log n)$ | unordered_set = hash $O(1)$ avg

reserve = capacity only | resize = actual elements